Milestone 6, Part 5a: Local Best Response

holdem-cfr walkthrough

Why another measurement

Part 4b ended with three head-to-head results that contradicted each other. One checkpoint beat a second, the second tied a third, and the third beat the first. Between strategies that are all near equilibrium, a match measures how one strategy's leaks line up with the other's, not how close either is to equilibrium. Ranking checkpoints needs the quantity that defines "close to equilibrium": **exploitability**, the amount a perfect counter-strategy would win.

On Kuhn, Leduc and the postflop subgames, exploitability was computed exactly with a best response over the whole tree. For the full no-limit game that is impossible on a laptop. **Local best response** (Lisý and Bowling, 2017) is the standard substitute. Its key property is that LBR is itself a strategy, so whatever it wins against the blueprint is a valid **lower bound** on the blueprint's exploitability, however crude LBR's own choices are. The bound can never overstate a weakness, only understate it.

How LBR decides

LBR is allowed to know the blueprint's strategy, but not its cards. At each of its decisions it does three things.

It tracks the opponent's range exactly. It starts with every hand not blocked by its own cards or the board. After each blueprint action, it multiplies every hand's weight by the probability that the blueprint, holding that hand, takes that action. This is Bayes' rule with the blueprint's actual strategy, and it is why LBR can find leaks: when the blueprint's actions reveal too much about its hand, LBR's range picture becomes sharp.

It computes equity against every hand in the range, assuming the hand is checked down. On the turn and river this is exact: 44 rivers, or a direct comparison. On the flop it samples 128 runouts.

It scores each action with a one-step lookahead:

- Fold: lose the chips already in. - Check or call: win or lose the matched pot at showdown, weighted by equity against the range. - Bet or raise to x: each hand in the range folds with the blueprint's own fold probability at the resulting node. The hands that continue are assumed to call, and LBR's equity is taken against that continuing range, which is the paper's formulation.

It then plays the best-scoring action. Preflop it always checks or calls, because a one-step lookahead is a poor guide with four streets to come. LBR also chooses only from the blueprint's own abstract actions. So this measures the leaks that the card abstraction and imperfect convergence create, not leaks from bet sizes the blueprint never saw, which is the "on-tree" setting in the paper.

Checks

- Equities match brute force exactly on the turn and river, and within 0.04 on the flop with sampling. - The range after a preflop limp equals the blueprint's limp probability for every hand, to 10^{-12}. - On a small made-hand-category blueprint, LBR wins **+7,492 ± 1,237** mbb/hand against the untrained (uniform) strategy and **+1,792 ± 707** against the trained one. LBR finds real leaks, and training shrinks them.

Results

Every checkpoint, 25,000 duplicate pairs (50,000 hands), the same deals for each:

Checkpoint	LBR wins (mbb/hand)
34M iterations (VM, 6 min, f32)	+1,143 ± 138
408M (Mac, f32)	+1,207 ± 133
924M (Mac, f32)	**+1,100 ± 146**
1,552M (Mac, f64, average reset at 924M)	+1,389 ± 128

For scale, folding every hand loses 750 mbb/hand. In the LBR paper, competition bots from 2013 to 2016 lost between about 400 and 1,050 mbb/hand to an LBR that could only fold or call, and between about 2,600 and 4,000 once it could also bet the pot or go all-in on the turn and river. These numbers are not directly comparable to ours: those bots played full 200 bb no-limit with off-tree bets allowed. They are the right order of magnitude for a blueprint without real-time search.

Two findings.

Training stopped helping after the first few minutes. 34M, 408M and 924M iterations sit within one confidence interval of each other. A 27-fold increase in training changed nothing LBR can detect. The leak LBR finds is not unfinished training. It is the **card abstraction**: 50 buckets per street, with no memory of earlier streets, mean that hands needing different play share one strategy. LBR's exact range tracking finds those hands. More iterations converge the blueprint toward the equilibrium of the *abstract* game, whose distance from the real game's equilibrium does not shrink.

Resetting the average made things worse. The 1,552M checkpoint, whose average strategy was rebuilt from 628 million f64 iterations after the reset, is the most exploitable of the four: 289 mbb/hand worse than 924M, outside the combined interval. The likely reason: the reset threw away the average at rarely visited nodes, deep river lines that 628 million iterations visit only a few times, and replaced it with a noisy one. The f32 damage was real, but it was concentrated at busy preflop nodes, where so many visits accumulate that the damage there evidently cost less than the reset did. The precision fix stays, because every future run needs it. The reset was the wrong cure for an old checkpoint. The web table goes back to the 924M blueprint, the least exploitable measured.

This is what LBR is for. Head-to-head said 1,552M ties 924M, and it could not rank the others at all. LBR ranks them, and its verdict points to where the next gain is: not more training, but **real-time search**. Re-solving the turn and river with the exact postflop solver, using the blueprint's ranges, replaces bucketed play with play on the exact cards, which is how DeepStack, Libratus and Pluribus closed this gap.

Reproduce

```
cargo run --release -p cli -- lbr --pairs 25000
cargo run --release -p cli -- lbr --checkpoints data/bp_408M.bin,data/bp_924M.bin,data/blueprint.bin --pai
cargo test --release -p hunl --test lbr -- --nocapture
```